CodingHub

Project documentation

React + Vite | Express + TypeScript | MongoDB

- Connect coding profiles
- Track contests
- Keep reminders

Implementation guide | Updated October 2026

A practical guide to what the application does, how its services fit together, how to run it locally, and how data is stored.

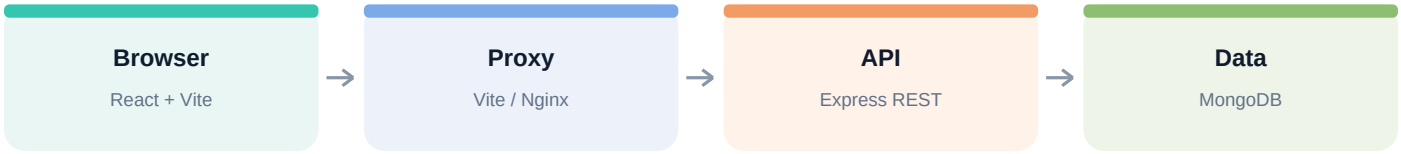
01 Project at a glance

CodingHub brings competitive-programming profiles, progress snapshots, contests, and personal contest reminders into one web application. Guests can browse public catalogs and contests. Registered users manage their own profile connections, dashboard, and reminders.

What it does

- **Accounts:** registration, login, bearer-token authentication, profile edits, and account deletion.
- **Platform profiles:** connect handles and sync public rating / solved statistics from Codeforces and LeetCode.
- **Contest tools:** search, filter, paginate, create, edit, delete, and save personal reminders.
- **Progress:** a per-user dashboard and a public leaderboard based on observed profile statistics.
- **Reliability:** MongoDB persistence, validation, standardized errors, rate limits, ETags, conditional updates, idempotent contest creation, and bounded retries.

System structure



Platform adapters call public Codeforces and LeetCode endpoints during sync.

The React client sends API requests through the development Vite proxy or the production Nginx proxy. Express routes handle authorization and validation, then call Mongoose-backed operations. Profile sync uses adapters for external public APIs.

Current provider coverage: automatic sync is implemented for Codeforces and LeetCode. CodeChef, HackerRank, and AtCoder are catalog entries without automatic statistics adapters. Contest data is managed through the API; no external contest feed is configured.

02 Core user workflows

Connect and sync a coding profile

1. Sign in and save a platform handle with `PUT /api/v1/platforms/{platformId}`.

2. Request a refresh with `POST /api/v1/platforms/{platformId}/sync`. The adapter fetches public data with a timeout and limited retries for transient failures.

3. Read the saved result from `GET /api/v1/platforms/connected` or `GET /api/v1/dashboard`. Unsupported providers and unknown handles do not receive invented statistics.

Authentication and account safety

- Passwords are stored as scrypt hashes. Signed bearer tokens identify the current user.
- Protected routes scope reads and writes to the authenticated account; deleting an account also removes owned profiles, reminders, contests, and idempotency records.
- Sensitive fields such as password hashes are excluded from public user projections.
- Errors use the Problem Details JSON format with status, detail, request ID, and a stable error code.

Safe repeated requests and updates

Contest creation accepts an optional `Idempotency-Key`. Repeating the same key and body replays the stored result; reusing it with a different body returns 409. Contest updates can include `If-Match`; a stale entity tag returns 412. Public reads provide ETags and support `If-None-Match`.

API conventions

- Base path: `/api/v1`. Protected routes require `Authorization: Bearer <token>`.
- JSON request validation rejects malformed or unknown fields where applicable; API errors use `application/problem+json`.
- Authenticated responses use private no-store caching. Public catalogs, contests, and leaderboard responses use ETags.

03 REST API reference

All paths below are relative to `/api/v1`. A lock indicates bearer authentication is required.

Area	Method and path	Use
Health	GET /health	Readiness check.
Auth	POST /auth/register POST /auth/login GET /auth/me	Create account, authenticate, or inspect current user. GET requires token.
Users	GET /users/me PATCH /users/me DELETE /users/me	Read/update current user or delete the account. All require token.
Platforms	GET /platforms GET /platforms/connected PUT /platforms/{platformId} POST /platforms/{platformId}/sync DELETE /platforms/{platformId}	Browse platform catalog; manage and sync own connections. Connected routes require token.
Dashboard	GET /dashboard	Read the signed-in user dashboard. Requires token.
Contests	GET /contests GET /contests/{contestId} POST /contests PATCH /contests/{contestId} DELETE /contests/{contestId}	List/detail are public. Create requires token; edit/delete require contest ownership.
Reminders	GET /contests/reminders PUT /contests/{contestId}/reminder DELETE /contests/{contestId}/reminder	Manage the signed-in user reminder list. Requires token.
Leaderboard	GET /leaderboard	Read the public aggregate leaderboard.

Useful query and header behavior

- Contest list query: `page`, `limit` (maximum 100), `platform`, `status`, and `search`.
- Contest create: optional `Idempotency-Key`; stored replay window is 24 hours.
- Contest update: optional `If-Match`; stale version returns 412.
- Public conditional read: send `If-None-Match`; unchanged data may return 304.

For exact schemas, fields, and status codes, consult `docs/openapi.yaml`.

04 Data model and persistence

MongoDB is the persistent store. Mongoose schemas validate documents and define indexes. User and contest IDs remain UUID strings so existing API references and imported relationships remain stable.

Collection	Key fields and relationships
Users	Name, normalized unique email, password hash, created timestamp.
Profiles	User ID, platform ID, handle, stats snapshot, sync status/time. Unique per user and platform.
Contests	Title, platform, start time, URL, status, creator ID, timestamps. Creator controls edits/deletion.
Reminders	User ID and contest ID. Unique for each user/contest pair.
Idempotency records	Request key, body fingerprint, stored status/body, expiration, processing lease. Durable across restarts.
Rate-limit buckets	Shared request window state stored in MongoDB for cross-process coordination.

Integrity and ownership

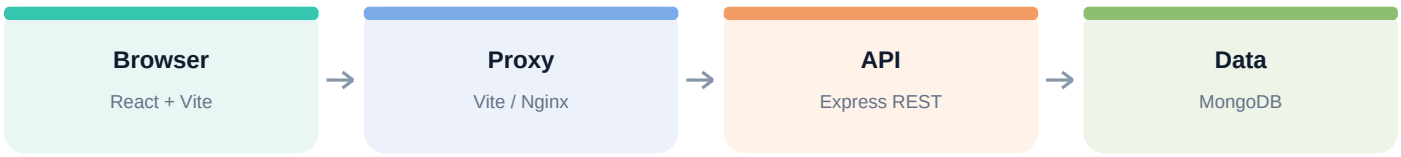
- Unique indexes protect account email, connected platform profile, reminders, and idempotency scope.
- Mongo queries handle contest filtering, sorting, and pagination; sensitive account fields are never returned to clients.
- Account removal cascades to its owned records and reminders referring to contests it created.
- Initial database connection and index setup complete before Express starts listening. Startup fails clearly when MongoDB cannot initialize.

JSON migration

The legacy `backend/data/store.json` remains an import source and is not edited or removed. Run `npm.cmd --prefix backend run migrate:json` with MongoDB configured. The importer uses insert-only upserts, preserves IDs and relationships, and prints collection totals without exposing account data. Rerunning does not duplicate records or overwrite existing Mongo documents. Keep the JSON file until you have verified the imported totals.

The local migration verified 4 users and 2 profile documents in MongoDB. The JSON source itself remains unchanged.

Data flow at runtime



Platform adapters call public Codeforces and LeetCode endpoints during sync.

05 Run the project locally

Requirements: Node.js 22.12+ (or 24+), npm, Docker Desktop, and Docker Compose.

```
npm.cmd --prefix frontend ci
npm.cmd --prefix backend ci
Copy-Item .env.example .env       # if root .env does not exist
Copy-Item backend/.env.example .env.local  # if missing
# Set private Mongo credentials and a random JWT_SECRET in .env.
# Match the Mongo credentials in .env.local's MONGODB_URI.
docker compose up -d mongo
npm.cmd run dev
```

Open the Vite URL printed in the terminal (normally http://localhost:5173). The frontend proxies `/api` to Express on port 5000. If 5173 is already occupied, Vite selects another port such as 5174.

MongoDB runs in Docker with a persistent named volume. The host port defaults to 27017 and binds to loopback. If 27017 is occupied, set `MONGO_PORT=27019` in root `.env` and update the host port in `.env.local` Mongo URI to match. The backend running on your computer connects to `localhost`; the API container connects to the Compose service name `mongo`.

Docker deployment

Run `docker compose up --build` for the Nginx-served frontend, Express API, and MongoDB. Compose waits for MongoDB health before starting the API, then waits for API health before starting the web container. Open `http://localhost:8080`. The named volume survives `docker compose down`; `docker compose down -v` removes it and deletes database contents.

Keep .env credentials private. Use unique random values for JWT_SECRET and the Mongo administrator password; do not commit local secret files.

06 Verification and operations

Useful project commands

```
npm.cmd --prefix backend run build
npm.cmd --prefix backend test
npm.cmd run lint
npm.cmd run build
npm.cmd --prefix backend run migrate:json
docker compose config
```

The backend test suite uses an isolated mongodb-memory-server database rather than the development database. Current verification: backend build passed; 6 backend tests passed; local Compose MongoDB started on host port 27019 because 27017 was occupied; the API answered through the Vite proxy; the migrated records remained available after restarting MongoDB.

Troubleshooting

Symptom	Check
MONGODB_URI is required	Create .env.local from backend/.env.example and set the host-run Mongo connection string.
ECONNREFUSED from Vite proxy	Confirm backend is listening on port 5000 and MongoDB is healthy. The API does not start if its initial Mongo connection fails.
Mongo port already in use	Pick a free MONGO_PORT in root .env and use that same localhost port in .env.local MONGODB_URI.
Mongo authentication error	Ensure root .env credentials match the host-run URI. Use URL-safe password characters or URL-encode reserved characters.
Port 5173 in use	Use the alternate Vite URL printed in the terminal, often http://localhost:5174.
Container API cannot reach Mongo	Use the service hostname mongo in the container URI; localhost refers to the API container itself.

Known limitations

- Only Codeforces and LeetCode have automatic profile synchronization adapters.
- Contest feed entries are currently supplied through the API; no third-party contest feed is configured.
- Live provider synchronization depends on upstream availability; automated tests exercise mocked retry behavior.
- The app is a single deployable API with modular routes and services; it is not split into microservices.

Project references

README.md: startup and operating instructions. docs/architecture.md: modules, relationships, caching, and API behavior. docs/openapi.yaml: machine-readable API contract. docs/audit.md: implementation decisions and verification record.